

Information Retrieval #6 — Index Compression

Severin Mills

27.03.2026



Inhaltsverzeichnis/Agenda

1. Solution Bonus Quiz
2. Lecture Recap
3. Bonus Exercise 5: Index Compression
4. Exam Questions
5. Kahoot

Bonus Quiz

Character ordering (e.g., alphabetical) allows strings to be compared. E.g., "*apple*" < "*ample*" < "*triple*". To add or lookup a term, which of the following statements is true?

- ☐ B-trees require character ordering; hash indices do not.
- ☐ Neither B-trees, nor hash indices require character ordering.
- ☐ B-trees do not require character ordering; hash indices do.
- ☐ Both B-trees and hash indices require character ordering.

Bonus Quiz

Character ordering (e.g., alphabetical) allows strings to be compared. E.g., "*apple*" < "*ample*" < "*triple*". To add or lookup a term, which of the following statements is true?

- ✓ B-trees require character ordering; hash indices do not.
- ☐ Neither B-trees, nor hash indices require character ordering.
- ☐ B-trees do not require character ordering; hash indices do.
- ☐ Both B-trees and hash indices require character ordering.

Bonus Quiz

From the following options, mark only those variations that would be stored in a Permuterm index for the term **hello**:

☐ \$hello

☐ llo\$he

☐ ll\$ohe

☐ ohe\$ll

Bonus Quiz

From the following options, mark only those variations that would be stored in a Permuterm index for the term **hello**:

✓ *\$hello*

✓ *llo\$he*

☐ *//\$ohe*

☐ *ohe\$ll*

Bonus Quiz

Complete the following table on a notepad to compute the edit distance between the words $s_1 = \text{"**pint**"}$ and $s_2 = \text{"**mind**"}$ (refer to the algorithm in Figure 3.5, page 59 of the book):

		p	i	n	t
	0	1	2	3	4
m	1				
i	2				
n	3				
d	4				

Bonus Quiz

Complete the following table on a notepad to compute the edit distance between the words $s_1 = \text{"*pint*"}$ and $s_2 = \text{"*mind*"}$ (refer to the algorithm in Figure 3.5, page 59 of the book):

		p	i	n	t
	0	1	2	3	4
m	1	1	2	3	4
i	2	2	1	2	3
n	3	3	2	1	2
d	4	4	3	2	2

Bonus Quiz

Let s_1 and s_2 be two strings. The dynamic programming (DP) table m is used to compute the Levenshtein edit distance between them. We use subscript i for the rows and j for the columns in the table.

a) Which of the following best describes the meaning of the table entry $m[i,j]$?

- ☐ The edit distance between the first $i-1$ characters of s_1 and the first $j-1$ characters of s_2
- ☐ The edit distance between the first i characters of s_1 and the first j characters of s_2
- ☐ The edit distance between the first $i+1$ characters of s_1 and the first $j+1$ characters of s_2
- ☐ None of the above

Bonus Quiz

Let s_1 and s_2 be two strings. The dynamic programming (DP) table m is used to compute the Levenshtein edit distance between them. We use subscript i for the rows and j for the columns in the table.

a) Which of the following best describes the meaning of the table entry $m[i,j]$?

- ☐ The edit distance between the first $i-1$ characters of s_1 and the first $j-1$ characters of s_2
- ✓ The edit distance between the first i characters of s_1 and the first j characters of s_2
- ☐ The edit distance between the first $i+1$ characters of s_1 and the first $j+1$ characters of s_2
- ☐ None of the above

Bonus Quiz

Let s_1 and s_2 be two strings. The dynamic programming (DP) table m is used to compute the Levenshtein edit distance between them. We use subscript i for the rows and j for the columns in the table.

b) Consider the initialization of the first *column* of the DP table. Which interpretation of $m[i,0] = i$ is correct?

- ☐ The cost of transforming an empty prefix of s_1 into the first j characters of s_2 using j insertions
- ☐ The cost of transforming an empty prefix of s_1 into the first j characters of s_2 using j deletions
- ☐ The cost of transforming the first i characters of s_1 into an empty prefix of s_2 using i insertions
- ☐ The cost of transforming the first i characters of s_1 into an empty prefix of s_2 using i deletions

Bonus Quiz

Let s_1 and s_2 be two strings. The dynamic programming (DP) table m is used to compute the Levenshtein edit distance between them. We use subscript i for the rows and j for the columns in the table.

b) Consider the initialization of the first *column* of the DP table. Which interpretation of $m[i,0] = i$ is correct?

- ☐ The cost of transforming an empty prefix of s_1 into the first j characters of s_2 using j insertions
- ☐ The cost of transforming an empty prefix of s_1 into the first j characters of s_2 using j deletions
- ☐ The cost of transforming the first i characters of s_1 into an empty prefix of s_2 using i insertions
- ✓ The cost of transforming the first i characters of s_1 into an empty prefix of s_2 using i deletions

Bonus Quiz

Let s_1 and s_2 be two strings. The dynamic programming (DP) table m is used to compute the Levenshtein edit distance between them. We use subscript i for the rows and j for the columns in the table.

c) Consider the initialization of the first *row* of the DP table. Which interpretation of $m[0,j] = j$ is correct?

- ☐ The cost of transforming the first i characters of s_1 into an empty prefix of s_2 using i insertions
- ☐ The cost of transforming the first i characters of s_1 into an empty prefix of s_2 using i deletions
- ☐ The cost of transforming an empty prefix of s_1 into the first j characters of s_2 using j deletions
- ☐ The cost of transforming an empty prefix of s_1 into the first j characters of s_2 using j insertions

Bonus Quiz

Let s_1 and s_2 be two strings. The dynamic programming (DP) table m is used to compute the Levenshtein edit distance between them. We use subscript i for the rows and j for the columns in the table.

c) Consider the initialization of the first *row* of the DP table. Which interpretation of $m[0,j] = j$ is correct?

- ☐ The cost of transforming the first i characters of s_1 into an empty prefix of s_2 using i insertions
- ☐ The cost of transforming the first i characters of s_1 into an empty prefix of s_2 using i deletions
- ☐ The cost of transforming an empty prefix of s_1 into the first j characters of s_2 using j deletions
- ✓ The cost of transforming an empty prefix of s_1 into the first j characters of s_2 using j insertions

Bonus Quiz

Calculating the edit distance between strings is expensive. A useful heuristic to estimate which strings are likely to have a small edit distance is the number of tri-grams (3-grams) they share.

In this example, consider the symbol \$ at the start and end of every word when computing tri-grams.

a) How many tri-grams are common between ***aababba*** and ***bbabaab***?

b) How many tri-grams does the string ***aababababba*** share with the string ***bbabababaab***?

Bonus Quiz

Calculating the edit distance between strings is expensive. A useful heuristic to estimate which strings are likely to have a small edit distance is the number of tri-grams (3-grams) they share.

In this example, consider the symbol \$ at the start and end of every word when computing tri-grams.

a) How many tri-grams are common between **aababba** and **bbabaab**?

$\{\$aa, aab, aba, bab, abb, bba, ba\$ \} \cap \{\$bb, bba, bab, aba, baa, aab, ab\$ \} = \{aab, aba, bab, bba\} = 4$

b) How many tri-grams does the string **aababababba** share with the string **bbabababaab**?

4

Bonus Quiz

Calculating the edit distance between strings is expensive. A useful heuristic to estimate which strings are likely to have a small edit distance is the number of tri-grams (3-grams) they share.

In this example, consider the symbol \$ at the start and end of every word when computing tri-grams.

Calculate the Jaccard coefficients for the following word pairs:

a) The Jaccard coefficient for $J(\textit{aababba}, \textit{bbabaab})$ is ...

b) The Jaccard coefficient for $J(\textit{aabababba}, \textit{bbababaab})$ is ...

Bonus Quiz

Calculating the edit distance between strings is expensive. A useful heuristic to estimate which strings are likely to have a small edit distance is the number of tri-grams (3-grams) they share.

In this example, consider the symbol \$ at the start and end of every word when computing tri-grams.

Calculate the Jaccard coefficients for the following word pairs:

a) The Jaccard coefficient for $J(\text{aababba}, \text{bbabaab})$ is ...

Intersection / Union, i.e. $4 / 6 = 2 / 3$

b) The Jaccard coefficient for $J(\text{aabababba}, \text{bbababaab})$ is ...

The same, $2 / 3$.

Bonus Quiz

Calculating the edit distance between strings is expensive. A useful heuristic to estimate which strings are likely to have a small edit distance is the number of tri-grams (3-grams) they share.

In this example, consider the symbol \$ at the start and end of every word when computing tri-grams.

Which of the following two typos is more likely to occur when trying to type **spray**, based on the Jaccard similarity index alone?

- spay
- pray

Bonus Quiz

Calculating the edit distance between strings is expensive. A useful heuristic to estimate which strings are likely to have a small edit distance is the number of tri-grams (3-grams) they share.

In this example, consider the symbol \$ at the start and end of every word when computing tri-grams.

Which of the following two typos is more likely to occur when trying to type **spray**, based on the Jaccard similarity index alone?

- spay
- pray

$$J(\text{spray}, \text{spay}) = \frac{\{\$sp,spr,pra,ray,ay\$ \} \cap \{\$sp,spa,pay,ay\$ \}}{\{\sp,spr,pra,ray,ay,spa,pay\}} = \frac{2}{7}$$

$$J(\text{spray}, \text{pray}) = \frac{\{\$sp,spr,pra,ray,ay\$ \} \cap \{\$pr,pra,ray,ay\$ \}}{\{\sp,spr,pra,ray,ay,pr\}} = \frac{3}{6}$$

So pray is more likely.

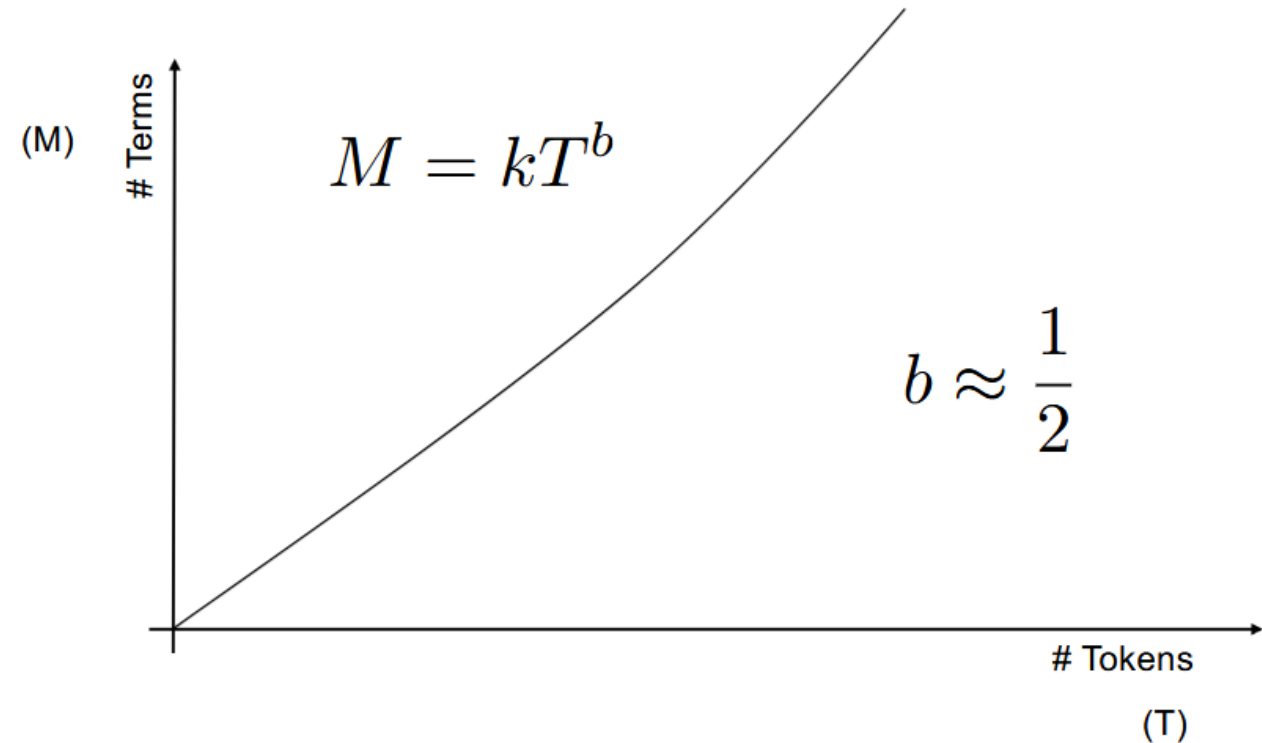
Term Statistics – Heap's Law

Let:

N: number of documents

T: number of (positional) tokens

M: number of terms (normalized types)



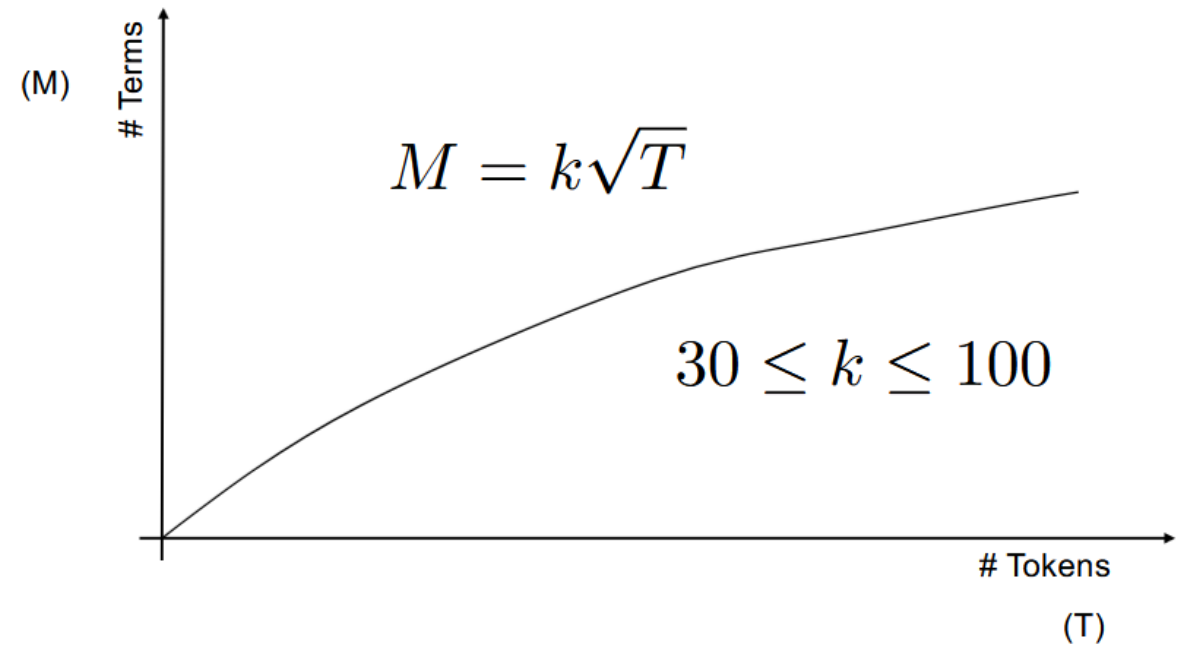
Term Statistics – Heap's Law

Let:

N: number of documents

T: number of (positional) tokens

M: number of terms (normalized types)



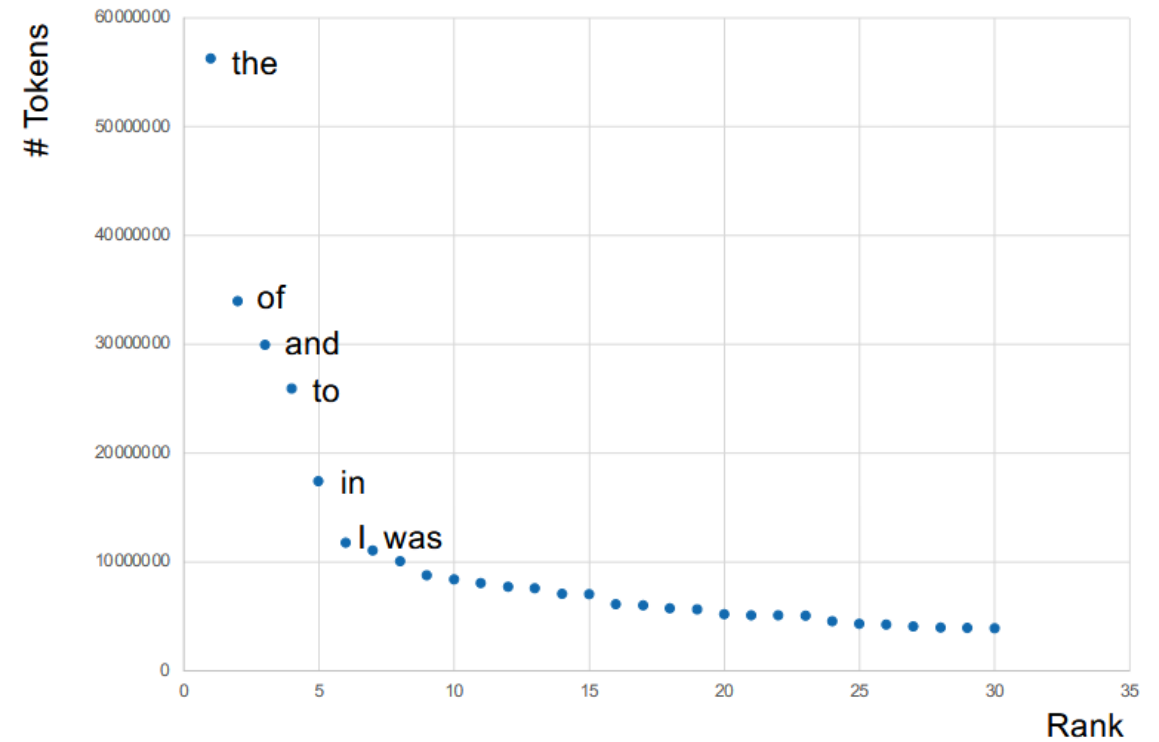
Distribution of Terms – Zipf's Law

Let:

N: number of documents

T: number of (positional) tokens

M: number of terms (normalized types)



Previous Compression Techniques

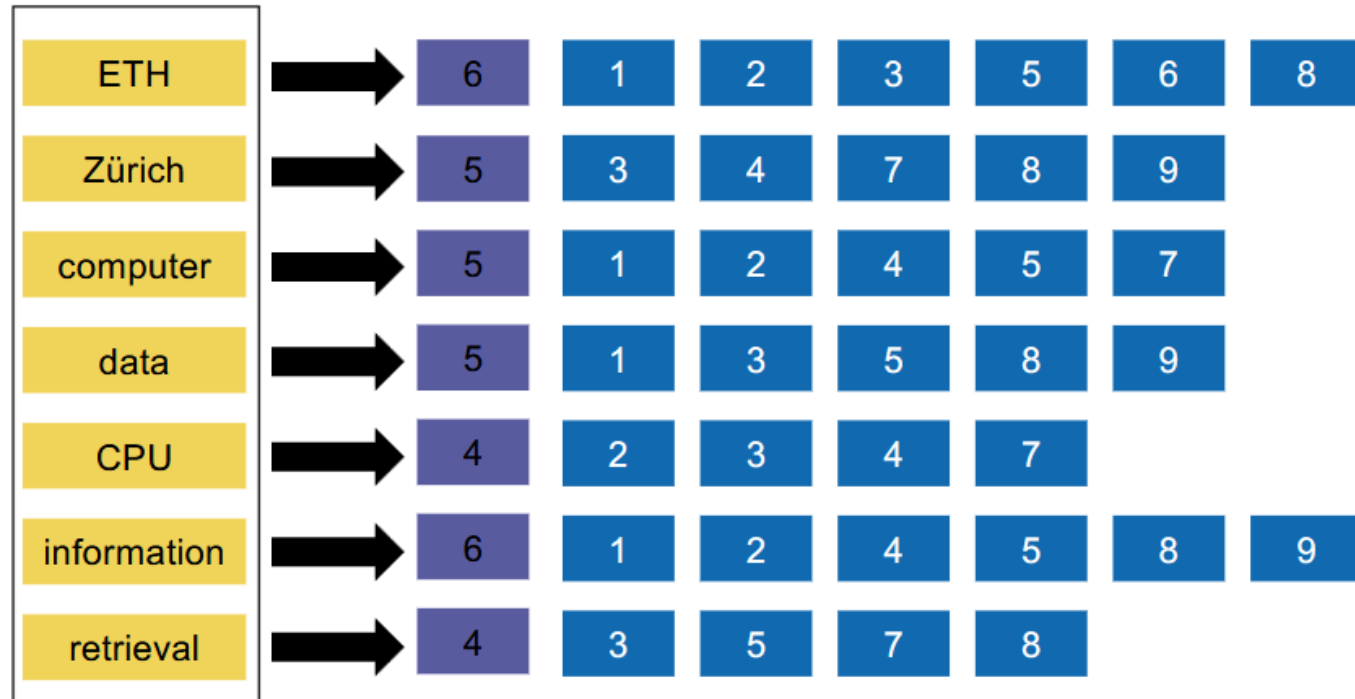
1 2 3 4 5	Remove numbers	-2%
Apple → apple	Case folding	-17%
and of the	Remove stopwords	-0%
computing → compute	Stemming	-17%



-33%

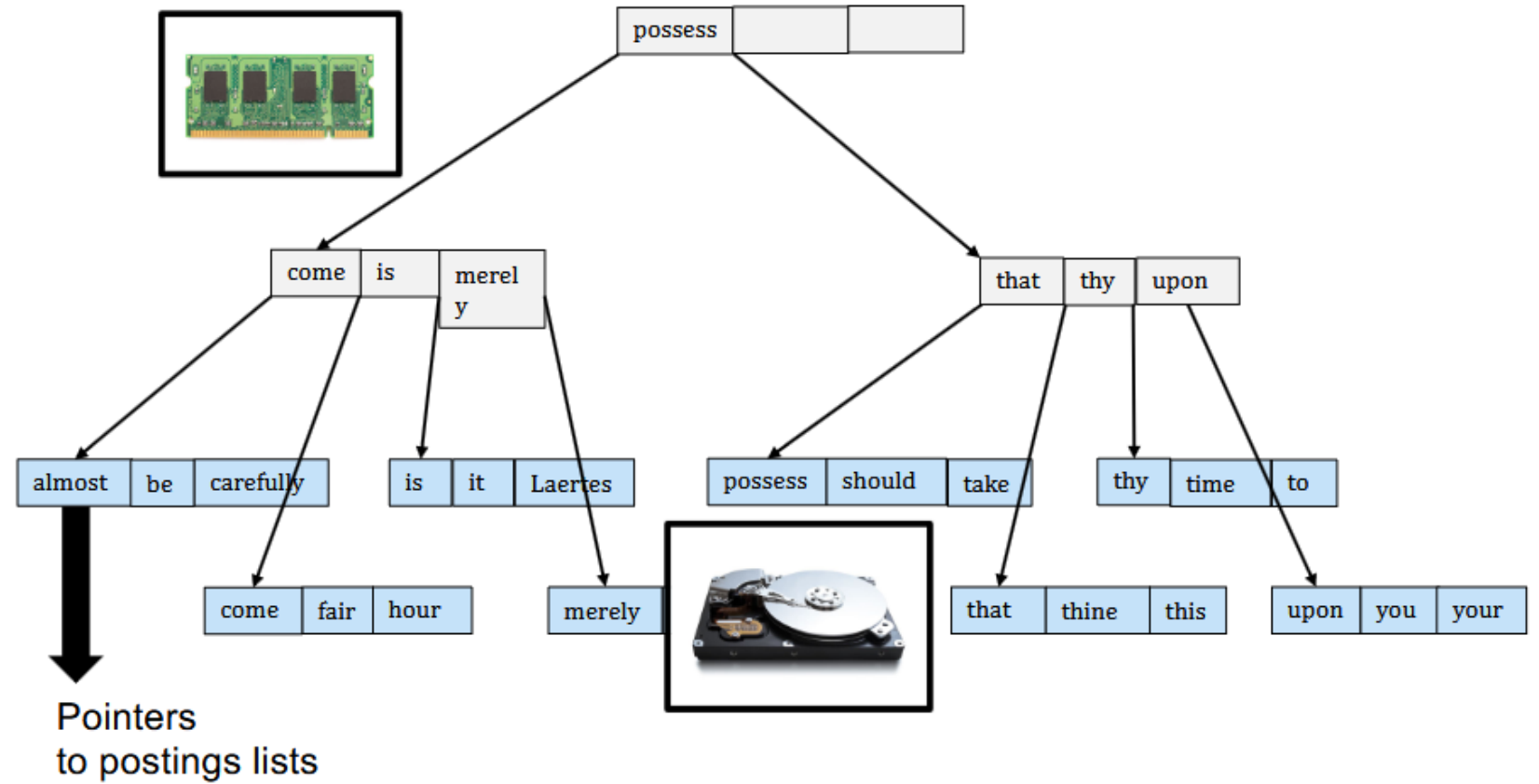
Our Goal

Compress everything!

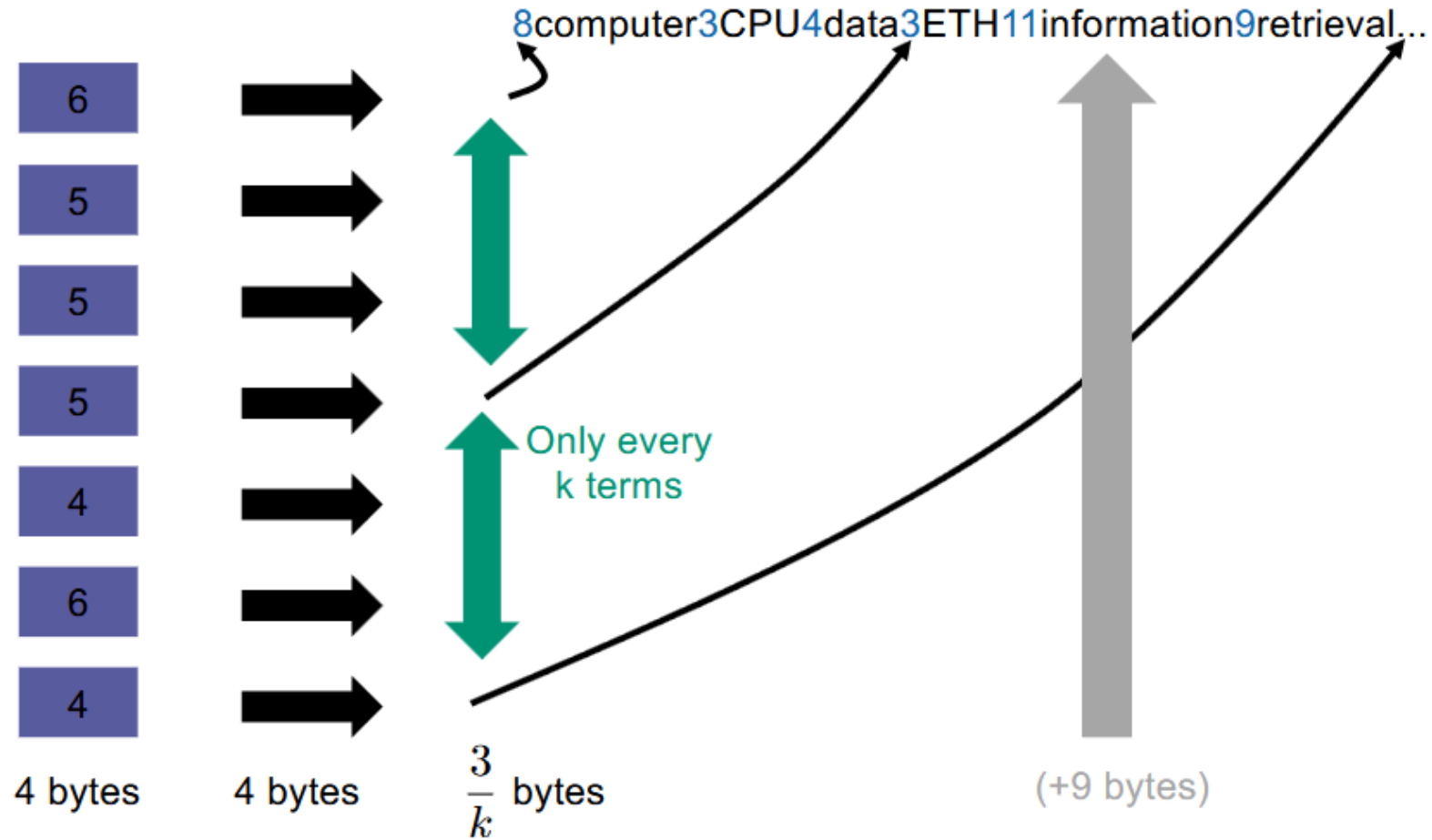


Compressing the dictionary

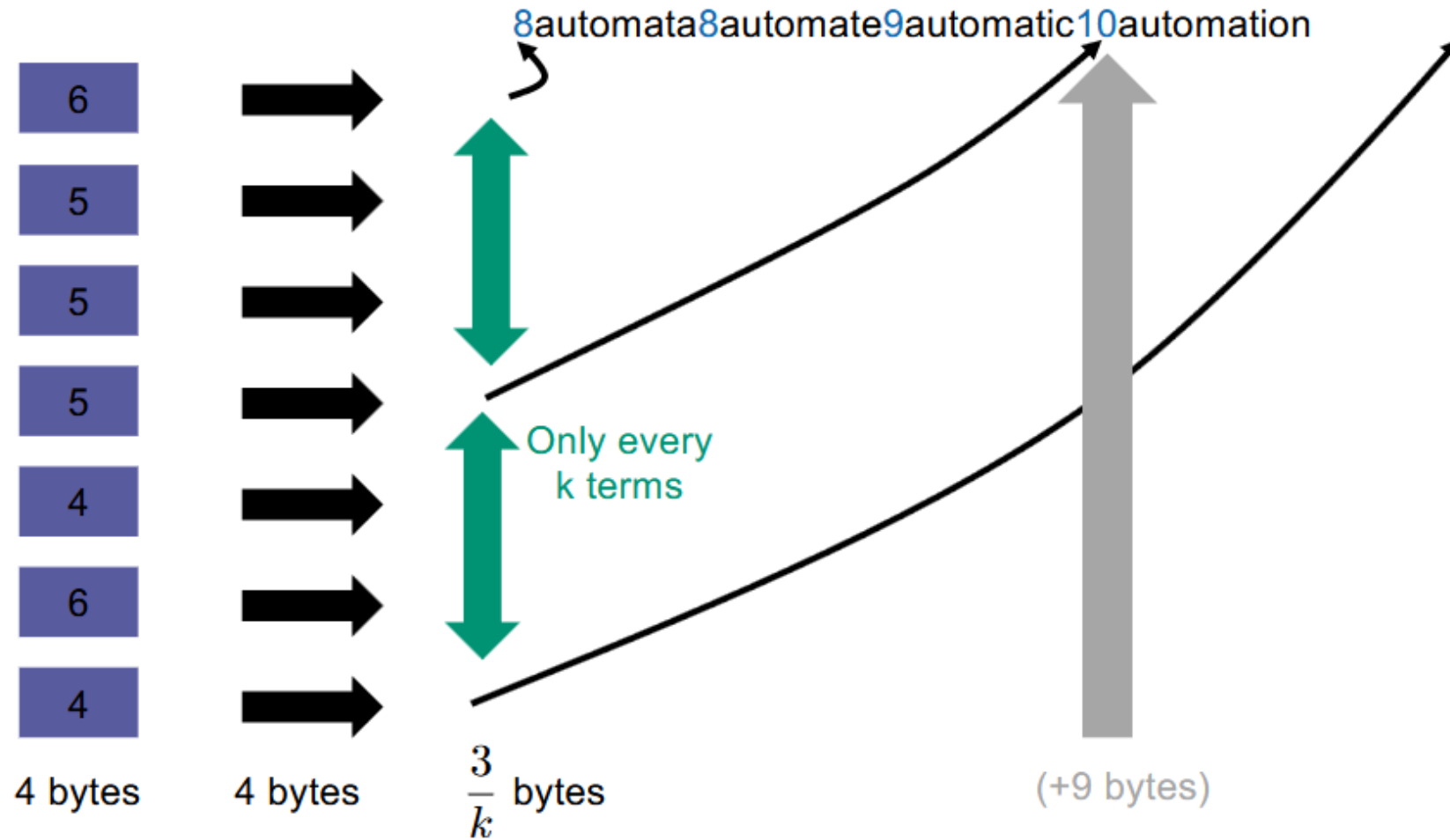
Currently: B+-Trees



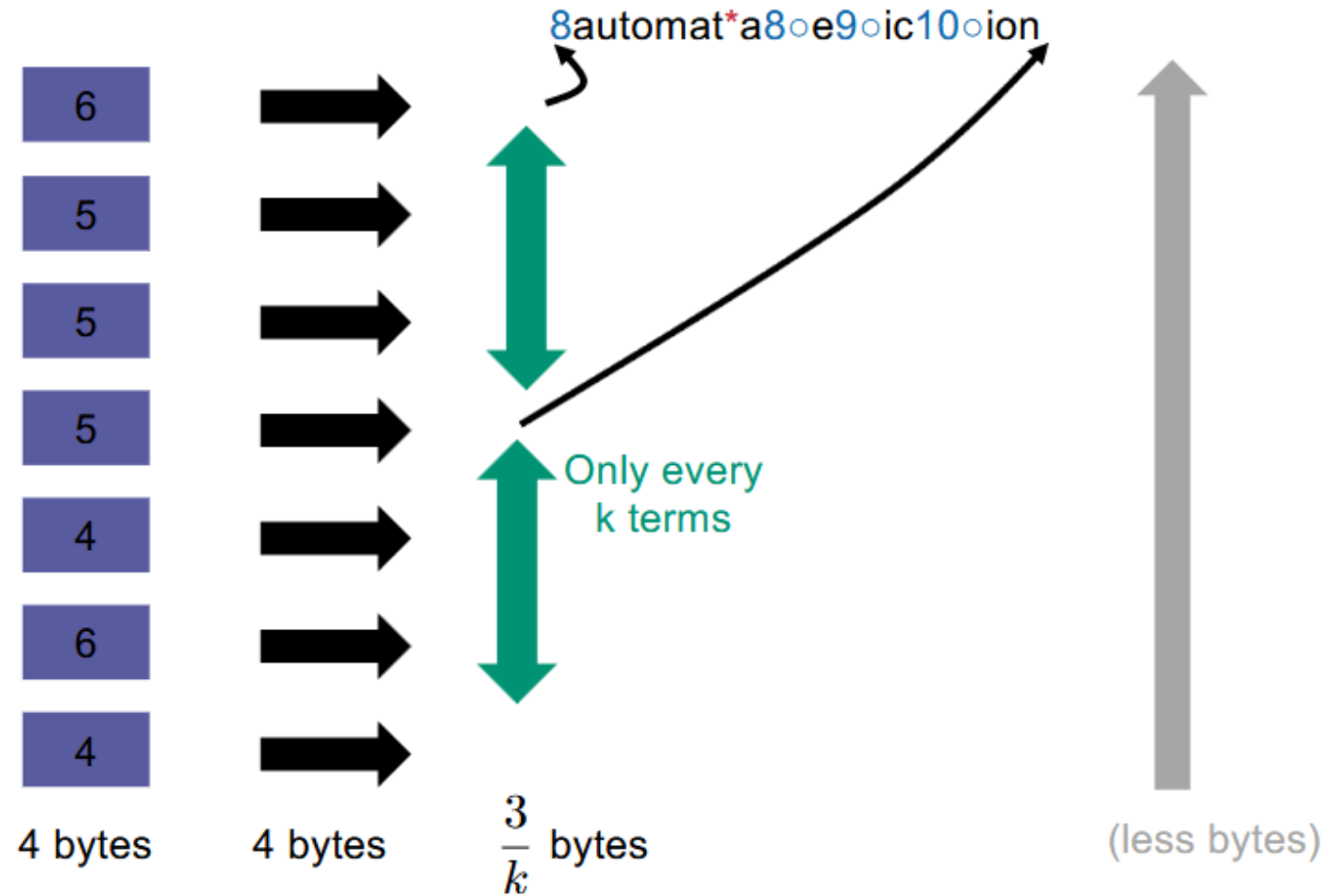
Idea 1: Blocked Storage



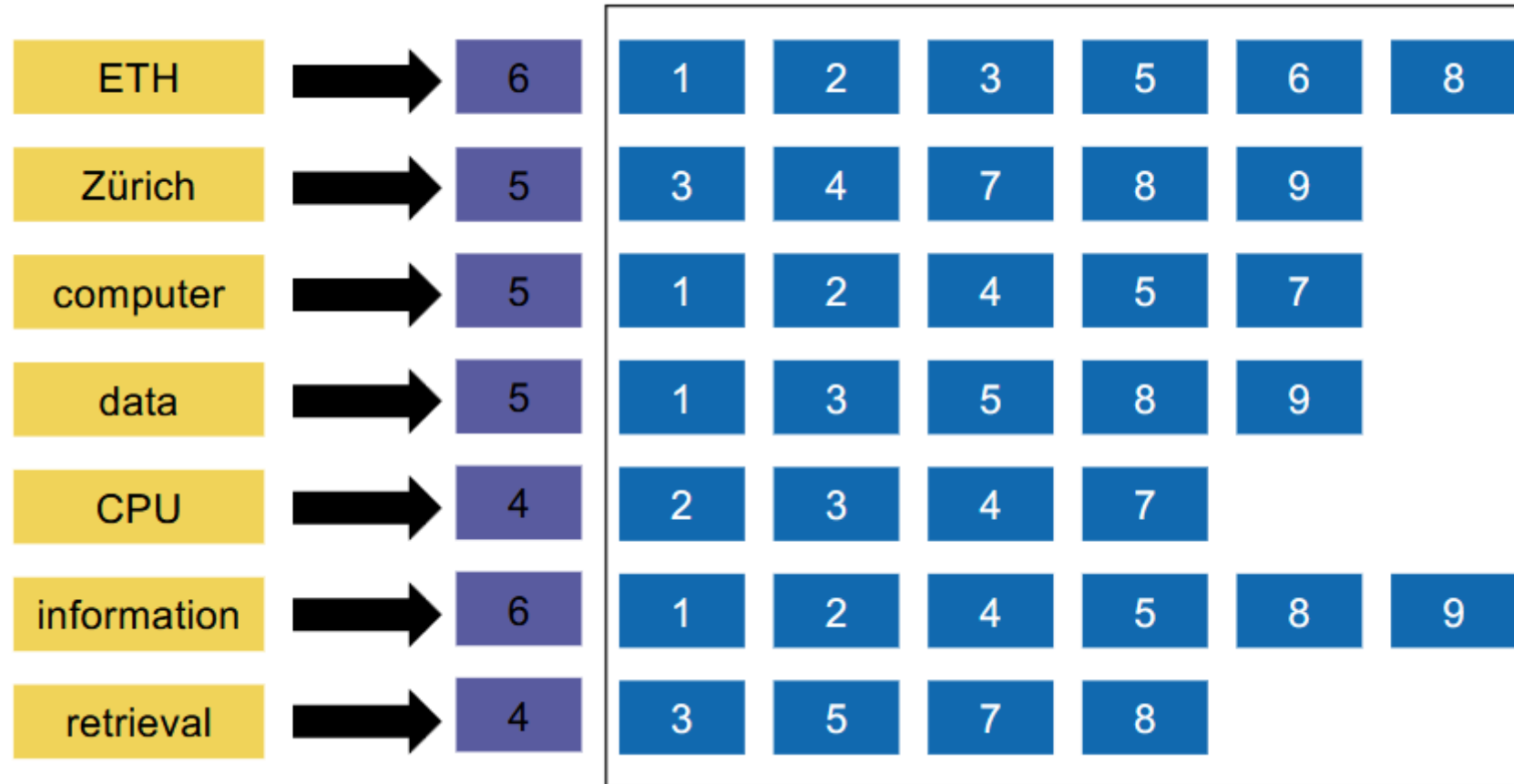
Idea 2: Front Coding



Idea 2: Front Coding

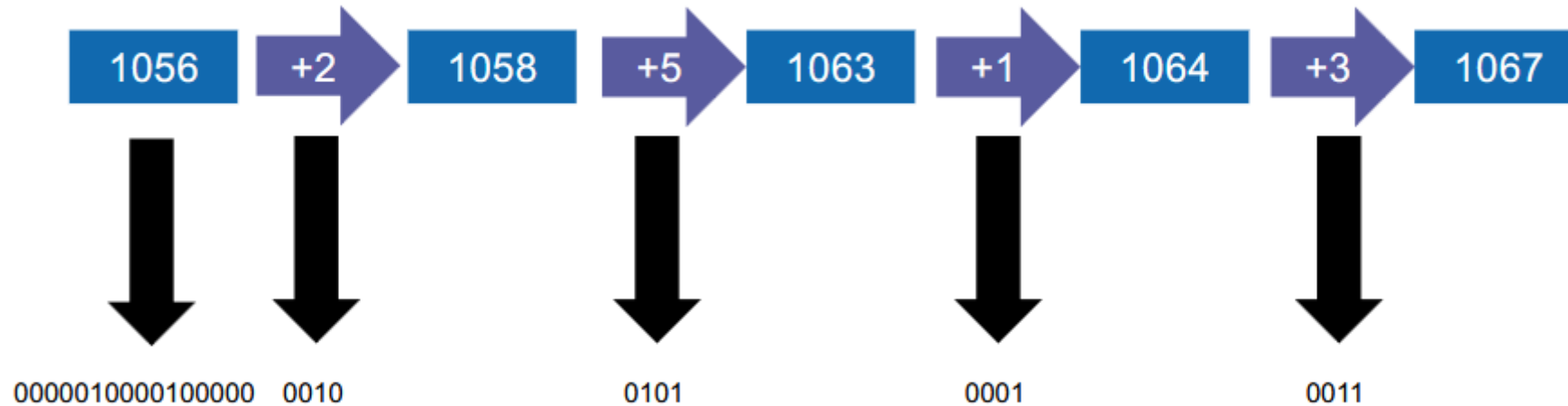


Encoding the Postings Lists



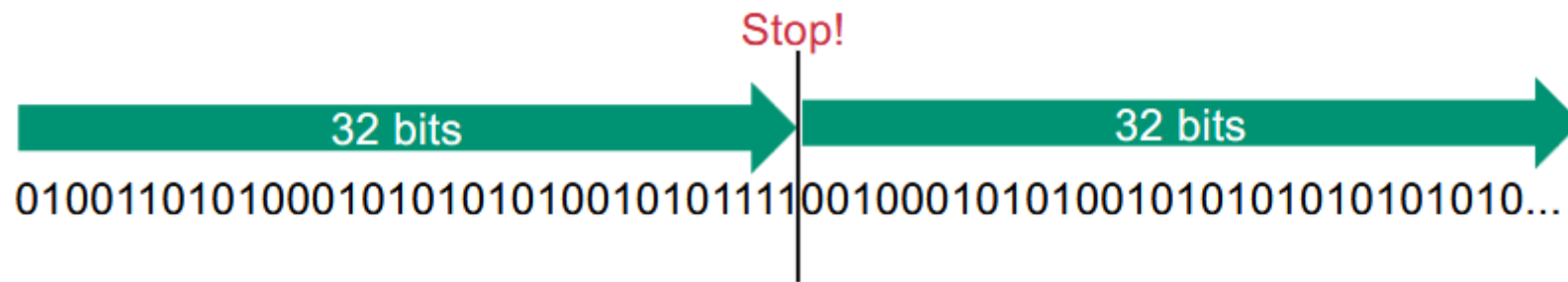
Encoding the Gaps

The encoding of the gaps are variable in length, depending on the size of the jump..



Fix Length Encoding

- Only works if we know where the boundaries are



Example: UTF-8

Character	Codepoint	Codepoint in binary	UTF-8 (variable length)
P	0050	1010 000 At most 7 bits	01010000
π	03C0	11 1100 0000 At most 11 bits	11001111 10000000
€	20AC	10 0000 1010 1100 At most 16 bits	11100010 10000010 10101100

Variable Byte Encoding (assume 8 bit packets)

0 0 0 0 0 0 0 0

continuation bit encoding on n-1 bits (here 7)

1: ends here

0: does not end here

Variable Byte Encoding

- Case 1: < 7 bits
 - Example: 4
 - Binary: 100
 - Extend: **0000**100
 - Add cont. bit: **1**0000100
- Case 2: $7 < x < 14$ bits
 - Example: 270
 - Binary: 1 0000 1110
 - Extend: **00000**10 0001110
 - Add cont. bit: **000000**10 **1**0001110
- And so on...

Gamma Encoding

- Unary Encoding:
 - e.g. 12 is 1111111111110

Example: 12

- Convert to Binary: 10011
- Remove leading 1: 0011
- Get length: 4
- Length in unary: 11110
- Together: 11110 0011

decimal	binary	binary without leading 1	length	length (unary)	gamma code
0	0	-	0		N/A
1	1	-	0		0
2	10	0	1	10	100
3	11	1	1	10	101
4	100	00	2	110	11000
5	101	01	2	110	11001
6	110	10	2	110	11010
7	111	11	2	110	11011
8	1000	000	3	1110	1110000
9	1001	001	3	1110	1110001
10	1010	010	3	1110	1110010
11	1011	011	3	1110	1110011
12	1100	100	3	1110	1110100
13	1101	101	3	1110	1110101
14	1110	110	3	1110	1110110
15	1111	111	3	1110	1110111
16	10000	0000	4	11110	111100000
17	10001	0001	4	11110	111100001
18	10010	0010	4	11110	111100010
19	10011	0011	4	11110	111100011
20	10100	0100	4	11110	111100100
21	10101	0101	4	11110	111100101
22	10110	0110	4	11110	111100110
23	10111	0111	4	11110	111100111
...	...	...	...	...	...
64	1000000	000000	6	1111110	1111110000000

Shannon Entropy

- The **Shannon Entropy** of a random variable X is the theoretical minimum average bits needed to encode.
- Gamma Encoding is near-optimal according to Shannon Entropy

for *any* distribution

$$\mathbb{E}[L_\gamma(X)] \leq \underline{3}H(X) = 3\mathbb{E}[I_X(X)]$$

one factor from optimal!

Exercise 5: Index Compression

- **Bonus Exercise**
 - Opens 27.03, 15:00 (i.e. now)
 - Closes 03.04, 15:00 (1 week from now)
 - 6/10 required to pass
 - Two attempts
 - Same concept as last time, theoretical + coding questions
- **Programming**
 - Encoding / Decoding using Gamma Code

Exam Questions

Assign the correct encoding method to these sequences of bits:

- 011 0101
 - 11111110
 - 0111 1101
 - 1111 0 1010
-
- Gamma code
 - 7-bit fixed-length encoding
 - Unary code
 - Variable-length encoding with 4-bit packets

Exam Questions

Assign the correct encoding method to these sequences of bits:

- 011 0101
 - 7-bit fixed-length encoding
- 11111110
- 0111 1101
- 1111 0 1010

- Gamma code
- Unary code
- Variable-length encoding with 4-bit packets

Exam Questions

Assign the correct encoding method to these sequences of bits:

- 011 0101
 - 7-bit fixed-length encoding
- 11111110
 - Unary code
- 0111 1101
- 1111 0 1010
- Gamma code
- Variable-length encoding with 4-bit packets

Exam Questions

Assign the correct encoding method to these sequences of bits:

- 011 0101
 - 7-bit fixed-length encoding
- 11111110
 - Unary code
- 0111 1101
 - Variable-length encoding with 4-bit packets
- 1111 0 1010
- Gamma code

Exam Questions

Assign the correct encoding method to these sequences of bits:

- 011 0101
 - 7-bit fixed-length encoding
- 11111110
 - Unary code
- 0111 1101
 - Variable-length encoding with 4-bit packets
- 1111 0 1010
 - Gamma code

Exam Questions

How many code points can we encode using the following encoding method:

12-bit variable-length encoding with 4-bit packets (not including shorter codes)

Exam Questions

How many code points can we encode using the following encoding method:

12-bit variable-length encoding with 4-bit packets (not including shorter codes)

0xxx 0xxx 1xxx

So excluding metadata, we can encode 9 bits, i.e. $2^9 = 512$ code points.

Exam Questions

How many code points can we encode using the following encoding method:

19-bit gamma code

Exam Questions

How many code points can we encode using the following encoding method:

19-bit gamma code

A 19-bit gamma code looks like this: 11111111 0 xxxxxxxx

Since we remove the leading 1, we can encode 1xxxxxxxx, i.e. $2^9 = 512$ code points.

Exam Questions

How many code points can we encode using the following encoding method:

12-bit fixed-length encoding

Exam Questions

How many code points can we encode using the following encoding method:

12-bit fixed-length encoding

No metadata, so $2^{12} = 4096$ code points.

Exam Questions

How many code points can we encode using the following encoding method:

UTF-8 up to 2 bytes

Exam Questions

How many code points can we encode using the following encoding method:

UTF-8 up to 2 bytes

2-byte UTF-8 encoding has the form: **110**xxxxx **10**xxxxxx, so we get $2^{11} = 2048$ code points.

1-byte UTF-8 encoding has the form: **0**xxxxxxx, so we get $2^7 = 128$ code points.

In total, we get: 2176 code points.

Exam Questions

Consider a collection with 10^6 tokens. Suppose the first 500 tokens result in a vocabulary size of 500 terms and the first 2500 tokens result in a vocabulary size of 1500 terms. If we apply Heap's law, what is the estimated number of terms in the whole collection?

Remember, Heap's law is $M = k \cdot \sqrt{T}$.

Exam Questions

Consider a collection with 10^6 tokens. Suppose the first 900 tokens result in a vocabulary size of 900 terms and the first 2500 tokens result in a vocabulary size of 1500 terms. If we apply Heap's law, what is the estimated number of terms in the whole collection?

Remember, Heap's law is $M = k \cdot \sqrt{T}$.

First, we want to find k . Fill in the values:

$$900 = k \cdot \sqrt{900} \Leftrightarrow 900 = k \cdot 30 \Leftrightarrow k = 30$$

$$1500 = k \cdot \sqrt{2500} \Leftrightarrow 1500 = k \cdot 50 \Leftrightarrow k = 30$$

So, we have a constant k . Then apply to the whole collection:

$$M = 30 \cdot \sqrt{10^6} \Leftrightarrow M = 30 \cdot 10^3 \Leftrightarrow M = 3000$$

Which means the whole collection will have 3000 terms.

Exam Questions

The most common word in a collection has a collection frequency of 4000. According to an application of Zipf's law, what would be the collection frequency of the fourth most common word in the collection.

Remember, Zipf's law is $\text{Frequency} = \frac{k}{n}$ where n is the rank.

Exam Questions

The most common word in a collection has a collection frequency of 4000. According to an application of Zipf's law, what would be the collection frequency of the fourth most common word in the collection.

Remember, Zipf's law is $\text{Frequency} = \frac{k}{n}$, where n is the rank.

Find k :

$$n = 1 \wedge \frac{k}{n} = 4000 \Rightarrow k = 4000$$

So, for $n = 4$:

$$\frac{k}{n} = \frac{4000}{4} = 1000$$

Kahoot

<https://create.kahoot.it/share/information-retrieval-index-compression/eab34273-e603-4ba3-a617-2a2130efabdb>